



MODUL 8 · STATE MANAGEMENT

Zustand: Store Factory & Integrasi Next.js

Store Factory Pattern untuk menghindari SSR data leaking.





Jangan: Global Store di Next.js SSR

Store global di module level = data User A bisa bocor ke User B di server. Security risk nyata.

✗ BERBAHAYA – global store di module level

```
// store/user-store.ts
import { create } from "zustand"

interface UserState {
  user: User | null
  setUser: (user: User | null) => void
}

// ✗ Global singleton – BERBAHAYA di SSR!
export const useUserStore = create<UserState>((set) =>
({user: null,
  setUser: (user) => set({ user }),
}))

// Kenapa berbahaya di Next.js?
// - Server Node.js proses semua request
// - Module di-evaluate SEKALI saat start server
// - Store ini SATU INSTANCE untuk semua request!
//
// Request User A: setUser(userA) → store.user = userA
// Request User B (langsung setelahnya):
//   store.user masih = userA! ← DATA BOCOR!
//
// Di client-side rendering = OK (per browser tab)
// Di SSR Next.js = SECURITY VULNERABILITY
```

✓ Store Factory – aman untuk SSR

```
// store/user-store.ts
import { createStore } from "zustand/vanilla"
import { useStore } from "zustand"

interface UserState {
  user: User | null
  setUser: (user: User | null) => void
}

// ✓ Factory: buat store BARU per request
export function createUserStore(
  initialUser: User | null = null
) {
  return createStore<UserState>((set) => ({
    user: initialUser,
    setUser: (user) => set({ user }),
  })))
}

// Tipe untuk referensi store
export type UserStore = ReturnType<typeof
createUserStore>
// Tidak ada export default store!
// Store dibuat dan dikirim via Provider
// → tiap request punya instance sendiri
```

S T A T E M A N A G E M E N T

Store Factory + Provider Pattern

Solusi resmi Zustand untuk Next.js App Router. Tiap request mendapat store instance baru.

providers/user-store-provider.tsx – pattern lengkap

```
"use client"
import { createContext, useContext, useRef, type ReactNode } from "react"
import { useStore } from "zustand"
import { createUserStore, type UserStore } from "@/store/user-store"
import type { UserState } from "@/store/user-store"

// 1. Context untuk distribute store
const UserStoreContext = createContext<UserStore | null>(null)

// 2. Provider: buat store SEKALI per component tree
export function UserStoreProvider({
  children,
  initialUser = null,
}: {
  children: ReactNode
  initialUser?: User | null
}) {
  // useRef: jangan buat store baru setiap render!
  const storeRef = useRef<UserStore | null>(null)
  if (!storeRef.current) {
    storeRef.current = createUserStore(initialUser)
  }
  return (
    <UserStoreContext.Provider value={storeRef.current}>
      {children}
    </UserStoreContext.Provider>
  )
}

// 3. Custom hook dengan selector
export function useUserStore<T>(selector: (state: UserState) => T): T {
  const store = useContext(UserStoreContext)
  if (!store) throw new Error("useUserStore must be used within UserStoreProvider")
  return useStore(store, selector)
}
```

Integrasi Provider di layout.tsx

Bungkus di layout — Server Component bisa pass initial data ke Store via Provider props.

app/layout.tsx + app/(dashboard)/layout.tsx — integrasi

```
// app/(dashboard)/layout.tsx — Server Component
// Fetch user di server, pass ke store sebagai initial state
import { cookies } from 'next/headers'
import { UserStoreProvider } from '@providers/user-store-provider'
import { getUserFromSession } from '@lib/auth'

export default async function DashboardLayout({
  children,
}): {
  children: React.ReactNode,
}) {
  // ✅ Fetch user di SERVER — aman, tidak perlu client
  const cookieStore = await cookies()
  const sessionToken = cookieStore.get('session')?.value
  const user = sessionToken ? await getUserFromSession(sessionToken) :
  null
  return (
    // ✅ Pass initial state dari server ke store
    <UserStoreProvider initialUser={user}>
      <nav>Dashboard Nav</nav>
      {children}
    </UserStoreProvider>
  )
}

// Client Component — pakai store dengan selector
// components/user-menu.tsx
;('use client')
import { useUserStore } from '@providers/user-store-provider'

export function UserMenu() {
  // Selector: hanya subscribe field 'user' — precise!
  const user = useUserStore((state) => state.user)
  return <div>{user?.name ?? 'Guest'}</div>
}
```


Zustand Middleware — persist & immer

persist: simpan state ke localStorage. immer: update state mutable-style.

persist middleware — cart yang survive refresh

```
import { create } from "zustand"
import { persist, createJSONStorage } from
"zustand/middleware"
interface CartState {
  items: CartItem[]
  addItem: (item: CartItem) => void
  removeItem: (id: string) => void
  clearCart: () => void
}

// ✅ Client-only store dengan persist
// Untuk cart tidak ada SSR concern
export const useCartStore = create<CartState>()(
  persist(
    (set) => ({
      items: [],
      addItem: (item) => set((state) => ({
        items: [...state.items, item],
      })),
      removeItem: (id) => set((state) => ({
        items: state.items.filter(i => i.id !== id),
      })),
      clearCart: () => set({ items: [] }),
    }),
    {
      name: "cart-storage", // localStorage key
      storage: createJSONStorage(() => localStorage),
    }
  )
)
```

Hydration fix untuk persist

```
// Masalah: SSR render {} tapi client
// load dari localStorage → hydration mismatch!

'use client'
import { useState, useEffect } from 'react'
import { useCartStore } from '@store/cart-store'

// ✅ Solusi: render after hydration
function CartIcon() {
  const [hydrated, setHydrated] = useState(false)
  const items = useCartStore((state) =>
state.items)
  useEffect(() => {
    setHydrated(true)
  }, [])

  // Saat SSR: tidak tampil (avoid mismatch)
  if (!hydrated) return <span>🛒</span>

  return <span>🛒 {items.length}</span>
}

// Alternatif lebih clean:
// Zustand punya built-in hydration helper:
// useCartStore.persist.hasHydrated()
```

Live Coding — Shopping Cart dengan Zustand

Store Factory untuk user state + persist store untuk cart. Integrasi lengkap ke Next.js.

01

`cart-store.ts`

Client-only store dengan persist. items: CartItem[], addItem/removeItem/clearCart. Tidak perlu Store Factory (tidak ada SSR concern).

02

`user-store.ts`

Store Factory (createUserStore). UserStoreProvider dengan useRef. Custom hook useUserStore dengan TypeScript-safe selector.

03

Layout Integration

Di app/(auth)/layout.tsx: fetch user di server, pass ke UserStoreProvider sebagai initialUser. Client Components baca via useUserStore.

04

Hydration Fix

Tambahkan hydration guard di CartIcon. Test: refresh halaman → cart masih ada. Test: server render → tidak ada hydration error.

Dua store pattern berbeda: cart-store = client-only (persist OK, tidak butuh factory). user-store = SSR-sensitive (butuh Store Factory + Provider). Pahami kapan pakai yang mana.

Evaluasi Pemahaman

Jawab sebelum lanjut ke slide berikutnya.

Q1 — Mengapa Zustand global store (module-level create()) berbahaya di Next.js SSR?

- A) Zustand tidak support TypeScript
- B) Store menjadi singleton — shared across requests. User A data bisa bocor ke User B.
- C) Performa lebih lambat dari Context

Q2 — Apa fungsi useRef di UserStoreProvider?

- A) Untuk menginisialisasi state awal saja
- B) Memastikan store dibuat SEKALI per Provider instance — tidak dibuat ulang setiap render
- C) Untuk menghindari TypeScript error

Q3 — Kenapa persist middleware bisa menyebabkan hydration mismatch?

- A) localStorage tidak tersedia di server — SSR render kosong, client load dari storage → mismatch
- B) Zustand versi terbaru tidak support persist
- C) Harus pakai sessionStorage, bukan localStorage

Tugas Mandiri

Implementasi full Zustand pattern yang aman untuk Next.js App Router.

01 `cart-store.ts`

Buat store dengan persist middleware. Interface CartItem { id, name, price, qty }. Actions: addItem, removeItem, updateQty, clearCart, total (selector).

02 `Store Factory`

Buat user-store.ts dengan createUserStore factory. Buat UserStoreProvider dengan useRef pattern. Export useUserStore hook dengan selector.

03 `Layout Integration`

Di app/layout.tsx: fetch user dari session (cookies), pass ke UserStoreProvider. Buat UserMenu Client Component yang baca dari store.

04 `Hydration Guard`

Tambahkan CartCounter yang handle hydration. Test: add items, refresh → items ada. Buka DevTools Console → tidak ada hydration warning.



Verifikasi security: buka dua browser tab berbeda → pastikan user state tidak tercampur.

Yang Sudah Kita Pelajari

- ✓ Global Zustand store = security risk di SSR. Data user bisa bocor antar requests.
- ✓ Store Factory pattern: createStore (bukan create) → Provider dengan useRef → custom hook.
- ✓ Server Component → fetch data → pass ke StoreProvider sebagai initialState props.
- ✓ persist middleware untuk localStorage sync. Handle hydration mismatch dengan guard.
- ✓ Decision: Context untuk simple shared UI state. Zustand untuk kompleks, async, atau perlu persist.

→ Selanjutnya: **Modul 9 — Database dengan Prisma 7 & PostgreSQL**

